UTS

PEMBELAJARAN MESIN



NAMA : Abhillasitta Dzakira Utami

NIM : 202431124

Kelas : H

Nama Dosen : Rizqia Cahyaningtyas, ST, M.Kom.

No. Bangku : 3

Nama Asisten : 1. Benedict Samuel Ardy

2. Ichwan Fauzi

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI 2
BAB I PENDAHULUAN 3
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 4
BAB II LANDASAN TEORI 5
2.1 Konsep Dasar Machine Learning 5
2.2 Regresi Linear dan Analisis Korelasi 6
2.2.1 Pengertian Regresi Linier 6
2.2.2 Metode Estimasi Least Squares 6
2.2.3 Diagram Pencar 7
2.2.4 Analisis Korelasi Pearson 7
2.2.5 Koefisien Determinasi 8
2.3 Decision Tree 8
2.4 K-Nearest Neighbors (KNN) 9
2.5 Naïve Bayes 10
BAB III HASIL & PEMBAHASAN 12
3.1 Gambaran Umum Dataset 12
3.2 Exploratory Data Analysis (EDA) 15
3.2.1 Analisis Distribusi Target Diabetes 17
3.2.2 Analisis Korelasi Antaratribut 20
3.3 Exploratory Data Analysis (EDA) 22
3.4 Pelatihan dan Evaluasi Model Klasifikasi 25
3.5 Pelatihan dan Evaluasi Model Model Linear Regression 29
BAB IV PENUTUP 32
DAFTAR PUSTAKA 34

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

1. Apa yang dimaksud dengan Regresi Linier dan bagaimana penerapan metode estimasi Least Squares dalam memprediksi nilai variabel dependen berdasarkan variabel independen?
2. Bagaimana analisis korelasi Pearson, diagram pencar (scatter plot), dan koefisien determinasi digunakan untuk mengevaluasi hubungan antarvariabel serta kualitas model regresi?
3. Bagaimana konsep Supervised Learning diterapkan dalam algoritma Decision Tree untuk menyelesaikan permasalahan klasifikasi dan regresi?
4. Bagaimana perhitungan Entropy, Information Gain, dan Gini Index digunakan dalam menentukan atribut terbaik untuk membentuk struktur pohon keputusan?
5. Bagaimana algoritma K-Nearest Neighbor menentukan kelas suatu data baru berdasarkan kedekatan jarak dengan data latih?

1.2 Tujuan Masalah

1. Memahami konsep Regresi Linier Sederhana serta penerapan metode Least Squares dalam membentuk persamaan regresi untuk melakukan prediksi.
2. Memahami penggunaan diagram pencar, korelasi Pearson, dan koefisien determinasi (\(R^2\)) dalam menganalisis hubungan antarvariabel dan mengevaluasi model regresi.
3. Menjelaskan konsep Supervised Learning serta penerapannya pada algoritma Decision Tree dalam permasalahan klasifikasi dan regresi.
4. Memahami proses pembentukan pohon keputusan melalui perhitungan Entropy, Information Gain, dan Gini Index.
5. Menjelaskan prinsip kerja algoritma KNN dalam menentukan klasifikasi data berdasarkan kedekatan jarak dan metode Majority Voting.

1.3 Manfaat Masalah

1. Menambah pemahaman mengenai konsep dasar Machine Learning dan berbagai algoritma yang digunakan dalam pengolahan data.
2. Memberikan pengetahuan mengenai prinsip matematika dan statistik yang mendasari Regresi Linier, Decision Tree, KNN, dan Naïve Bayes.
3. Menjadi sumber pembelajaran mengenai metode analisis, klasifikasi, prediksi, serta evaluasi model Machine Learning.
4. Memperluas wawasan mengenai perkembangan penerapan algoritma Machine Learning berdasarkan penelitian ilmiah yang telah dipublikasikan.
5. Menjadi dasar dalam melakukan implementasi algoritma menggunakan bahasa pemrograman maupun perangkat lunak pengolahan data.

BAB II
LANDASAN TEORI

2.1 Konsep Dasar Machine Learning

Machine Learning merupakan salah satu cabang dari kecerdasan buatan (Artificial Intelligence) yang memungkinkan komputer mempelajari pola dan hubungan dari sekumpulan data untuk menghasilkan prediksi, klasifikasi, maupun keputusan tanpa harus diprogram secara eksplisit untuk setiap permasalahan.

Dalam proses pembelajarannya, algoritma Machine Learning menggunakan data sebagai dasar untuk membangun model. Model tersebut kemudian dapat digunakan untuk menganalisis data baru berdasarkan pola yang telah dipelajari sebelumnya. Secara umum, Machine Learning dibagi menjadi beberapa jenis pembelajaran, yaitu:

1. Supervised Learning

   Supervised Learning merupakan metode pembelajaran mesin yang menggunakan data berlabel (labeled data), yaitu data yang telah memiliki informasi atau jawaban yang diketahui. Model dilatih untuk mempelajari hubungan antara atribut masukan (input) dengan hasil keluaran (output).

   Supervised Learning umumnya digunakan untuk dua jenis permasalahan, yaitu:

   - Klasifikasi (Classification): proses memprediksi kategori atau kelas suatu data, misalnya menentukan apakah suatu email termasuk spam atau bukan spam.
   - Regresi (Regression): proses memprediksi nilai numerik atau kontinu, misalnya memperkirakan harga rumah berdasarkan luas bangunan dan lokasi.

2. Unsupervised Learning

   Unsupervised Learning merupakan metode pembelajaran mesin yang menggunakan data tanpa label atau tanpa jawaban yang telah ditentukan sebelumnya. Algoritma berusaha menemukan pola, hubungan, maupun kelompok tertentu berdasarkan karakteristik data.

   Salah satu penerapannya adalah clustering, yaitu mengelompokkan data berdasarkan kemiripan karakteristik tanpa menentukan kelas terlebih dahulu.

Perbedaan utama kedua metode tersebut terletak pada keberadaan label dalam data pelatihan. Supervised Learning menggunakan data yang memiliki target, sedangkan Unsupervised Learning berusaha menemukan struktur atau pola tersembunyi dari data yang tersedia.

2.2 Regresi Linear dan Analisis Korelasi

2.2.1 Pengertian Regresi Linier

Regresi Linier merupakan salah satu metode statistik dan algoritma Machine Learning yang digunakan untuk menganalisis hubungan antara variabel independen (X) dengan variabel dependen (Y). Metode ini bertujuan untuk memperkirakan atau memprediksi nilai variabel dependen berdasarkan perubahan nilai variabel independen. Regresi Linier Sederhana melibatkan satu variabel independen dan satu variabel dependen. Hubungan kedua variabel tersebut digambarkan melalui sebuah garis lurus yang diperoleh berdasarkan pola data yang tersedia.

Persamaan umum Regresi Linier Sederhana adalah:

$$\boldsymbol{Y = a + bX}$$

Keterangan:

- $\boldsymbol{Y}$ = Variabel dependen atau variabel yang diprediksi.
- $\boldsymbol{X}$ = Variabel independen atau variabel yang memengaruhi prediksi.
- $\boldsymbol{a}$ = Konstanta atau nilai Y ketika X bernilai nol.
- $\boldsymbol{b}$ = Koefisien regresi atau perubahan nilai Y untuk setiap satu satuan perubahan X.

Koefisien regresi menunjukkan arah hubungan antara kedua variabel. Apabila nilai $\boldsymbol{b}$ positif, hubungan kedua variabel bersifat searah. Sebaliknya, apabila nilai $\boldsymbol{b}$ negatif, hubungan kedua variabel bersifat berlawanan arah.

2.2.2 Metode Estimasi Least Squares

Metode Least Squares atau metode kuadrat terkecil merupakan teknik yang digunakan untuk menentukan garis regresi terbaik dengan meminimalkan jumlah kuadrat selisih antara nilai aktual dan nilai prediksi. Selisih antara nilai aktual dengan nilai prediksi disebut residual atau error. Rumus fungsi objektif metode Least Squares adalah:

$$SSE = \sum_{i=1}^{n} (Y_i - \widehat{Y_i})$$

Keterangan :

- SSE = Sum of Squared Errors.
- Y_i = Nilai aktual.
- Y_i= Nilai prediksi.
- n = Jumlah data.

2.2.3 Diagram Pencar

Scatter Plot merupakan grafik yang digunakan untuk menggambarkan hubungan antara dua variabel numerik. Setiap titik pada grafik menunjukkan pasangan nilai dari variabel X dan Y. Diagram pencar dapat digunakan untuk mengidentifikasi pola hubungan antarvariabel sebelum membangun model regresi. Pola hubungan dalam diagram pencar dapat dibedakan menjadi tiga jenis:

1. Korelasi Positif: titik-titik cenderung membentuk pola dari kiri bawah menuju kanan atas.
2. Korelasi Negatif: titik-titik cenderung membentuk pola dari kiri atas menuju kanan bawah.
3. Tidak Berkorelasi: titik-titik tersebar tanpa menunjukkan pola hubungan yang jelas.

2.2.4 Analisis Korelasi Pearson

Korelasi Pearson merupakan metode statistik yang digunakan untuk mengukur kekuatan dan arah hubungan linier antara dua variabel numerik. Nilai koefisien korelasi Pearson berada pada rentang -1 sampai +1.
Rumus korelasi Pearson adalah:

$$r = \frac{n\sum XY - \sum X \sum Y}{\sqrt{[n\sum X^2 - (\sum X)^2][n\sum Y^2 - (\sum Y)^2]}}$$

Nilai absolut r	Interpretasi
0,00 – 0,19	Sangat Rendah
0,20 – 0,39	Rendah
0,40 – 0,59	Sedang

0,60 – 0,79	Kuat
0,80 – 1,00	Sangat Kuat

2.2.5 Koefisien Determinasi

Koefisien determinasi atau R^2 merupakan ukuran yang digunakan untuk mengetahui seberapa besar variasi variabel dependen dapat dijelaskan oleh model regresi. Rumus koefisien determinasi adalah:

$$\boldsymbol{R^2 = 1 - \frac{SSE}{SST}}$$

Keterangan:

- $\boldsymbol{R^2}$ = Koefisien determinasi.
- $\boldsymbol{SSE}$= Jumlah kuadrat kesalahan prediksi.
- $\boldsymbol{SST}$ = Jumlah kuadrat total variasi data.

2.3 Decision Tree

Decision Tree atau pohon keputusan merupakan algoritma Machine Learning yang digunakan untuk menyelesaikan permasalahan klasifikasi dan regresi dengan membentuk struktur menyerupai pohon. Algoritma ini bekerja dengan membagi data ke dalam beberapa kelompok berdasarkan atribut atau fitur tertentu. Setiap proses pembagian dilakukan untuk memperoleh kelompok data yang memiliki karakteristik semakin homogen. Struktur Decision Tree terdiri atas beberapa komponen utama:

1. Root Node (Akar): simpul pertama yang mewakili seluruh data sebelum dilakukan pembagian.
2. Decision Node: simpul yang menunjukkan proses pengambilan keputusan berdasarkan atribut tertentu.
3. Branch (Cabang): garis penghubung yang menunjukkan hasil dari suatu keputusan atau kondisi.
4. Leaf Node (Daun): simpul akhir yang menghasilkan keputusan atau prediksi.

Decision Tree memiliki kelebihan berupa struktur yang relatif mudah dipahami dan hasil prediksi yang dapat dijelaskan melalui aturan keputusan.

Proses pembentukan pohon keputusan dilakukan melalui beberapa tahapan berikut:

1. Menyiapkan data latih (training data) yang memiliki atribut dan label atau target.
2. Menghitung tingkat kemurnian data berdasarkan metode tertentu.
3. Menghitung nilai setiap atribut menggunakan Entropy, Information Gain, atau Gini Index.
4. Memilih atribut terbaik sebagai titik pembagian (split).
5. Membagi data berdasarkan atribut yang telah dipilih.
6. Mengulangi proses pembagian hingga memenuhi kondisi penghentian.
7. Menghasilkan pohon keputusan yang dapat digunakan untuk memprediksi data baru

Dalam menentukan atribut terbaik untuk melakukan pemisahan data, Decision Tree dapat menggunakan beberapa metode, antara lain:

a. Entropy

   Entropy digunakan untuk mengukur tingkat ketidakpastian atau ketidakmurnian dalam suatu kelompok data. Semakin tinggi nilai entropy, semakin beragam kelas yang terdapat dalam kelompok tersebut. Sebaliknya, nilai entropy yang rendah menunjukkan bahwa data cenderung berasal dari kelas yang sama.

b. Information Gain

   Information Gain digunakan untuk mengetahui seberapa besar pengurangan ketidakpastian setelah data dibagi berdasarkan suatu atribut. Atribut dengan nilai Information Gain yang lebih tinggi umumnya dianggap lebih baik untuk digunakan sebagai dasar percabangan.

c. Gini Index

   Gini Index merupakan metode lain yang digunakan untuk mengukur tingkat ketidakmurnian data. Nilai Gini yang lebih rendah menunjukkan bahwa kelompok data memiliki distribusi kelas yang lebih homogen.

2.4 K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) merupakan algoritma Machine Learning yang digunakan untuk melakukan klasifikasi maupun regresi berdasarkan kedekatan jarak antarobjek dalam suatu dataset. Algoritma ini termasuk metode Supervised Learning

yang menggunakan data pelatihan sebagai acuan untuk menentukan hasil prediksi terhadap data baru.

Prinsip utama KNN adalah bahwa data yang memiliki karakteristik serupa cenderung berada pada posisi yang berdekatan dalam ruang fitur. Oleh karena itu, proses prediksi dilakukan dengan mencari sejumlah tetangga terdekat dari data yang akan diklasifikasikan. Dalam penelitian ini, KNN digunakan untuk mengklasifikasikan status diabetes berdasarkan kemiripan karakteristik individu dengan data yang telah tersedia. KNN memiliki tahapan kerja yang relatif sederhana, yaitu:

1. Menentukan nilai K sebagai jumlah tetangga terdekat yang akan digunakan.
2. Menghitung jarak antara data yang akan diprediksi dengan seluruh data pelatihan.
3. Mengurutkan data berdasarkan jarak terdekat.
4. Memilih sejumlah K data dengan jarak paling kecil.
5. Menentukan hasil klasifikasi berdasarkan kelas yang paling banyak muncul di antara tetangga terdekat.

Sebagai contoh, apabila nilai K ditentukan sebesar 5, maka model akan mempertimbangkan lima data terdekat untuk menentukan kelas dari data baru Nilai K merupakan salah satu parameter penting dalam algoritma KNN karena dapat memengaruhi hasil klasifikasi.. Nilai K yang terlalu kecil dapat menyebabkan model lebih sensitif terhadap noise atau data yang tidak representatif. Sebaliknya, nilai K yang terlalu besar dapat membuat model kurang mampu menangkap pola lokal dari data. Oleh karena itu, pemilihan nilai K perlu dilakukan melalui pengujian beberapa alternatif nilai untuk memperoleh hasil klasifikasi yang optimal.

2.5 Naïve Bayes

Naive Bayes merupakan algoritma klasifikasi yang menggunakan prinsip probabilitas berdasarkan Teorema Bayes. Algoritma ini memperkirakan kemungkinan suatu data termasuk ke dalam kelas tertentu berdasarkan karakteristik atau atribut yang dimilikinya. Disebut naive karena algoritma ini menggunakan asumsi bahwa setiap atribut bersifat saling independen terhadap atribut lainnya ketika kelas data telah diketahui. Meskipun asumsi tersebut tidak selalu sepenuhnya sesuai dengan kondisi nyata, Naive Bayes tetap banyak digunakan karena proses perhitungannya relatif sederhana dan efisien.

Dalam penelitian ini, Naive Bayes digunakan untuk memprediksi klasifikasi status diabetes berdasarkan karakteristik individu yang terdapat dalam dataset. Naive Bayes bekerja dengan menghitung probabilitas suatu data terhadap setiap kemungkinan kelas. Selanjutnya, model memilih kelas dengan probabilitas paling tinggi sebagai hasil prediksi. Terdapat beberapa komponen utama yang digunakan dalam perhitungan Naive Bayes, yaitu:

1. Prior Probability

   Prior Probability merupakan probabilitas awal dari suatu kelas sebelum mempertimbangkan karakteristik data. Contohnya adalah proporsi jumlah individu yang termasuk dalam kategori diabetes dan tidak diabetes.
2. Likelihood

   Likelihood merupakan probabilitas munculnya suatu karakteristik berdasarkan kelas tertentu. Komponen ini digunakan untuk mengetahui seberapa mungkin suatu atribut ditemukan pada kelompok tertentu.
3. Posterior Probability

   Posterior Probability merupakan probabilitas akhir suatu kelas setelah mempertimbangkan informasi atau karakteristik yang dimiliki oleh data.

Naive Bayes memiliki beberapa jenis yang dapat digunakan sesuai dengan karakteristik data, di antaranya:

1. Gaussian Naive Bayes

   Gaussian Naive Bayes digunakan untuk atribut numerik yang diasumsikan mengikuti distribusi normal atau distribusi Gaussian. Metode ini sesuai untuk data berbentuk angka, seperti usia, BMI, dan kadar glukosa darah.
2. Multinomial Naive Bayes

   Multinomial Naive Bayes umumnya digunakan untuk data berbentuk frekuensi atau jumlah, seperti jumlah kemunculan kata dalam analisis teks.
3. Bernoulli Naive Bayes

   Bernoulli Naive Bayes digunakan untuk atribut biner yang hanya memiliki dua kemungkinan nilai, seperti ya atau tidak, benar atau salah, dan 0 atau 1.

Dalam penelitian ini, Gaussian Naive Bayes digunakan sebagai salah satu algoritma klasifikasi karena dataset memiliki sejumlah atribut numerik yang dapat dimanfaatkan dalam proses pembelajaran model.

BAB III
HASIL & PEMBAHASAN

3.1 Gambaran Umum Dataset

Menggunakan dataset diabetes yang diberi nama 02_diabetes.csv. Dataset tersebut diolah menggunakan bahasa pemrograman Python dengan bantuan beberapa library, seperti Pandas, NumPy, Matplotlib, dan Scikit-learn. Berdasarkan hasil pembacaan dataset, diperoleh sebanyak 100.000 baris data dengan 9 kolom atribut. Dataset tersebut terdiri atas informasi karakteristik individu, kondisi kesehatan, riwayat merokok, serta status diabetes.

No.	Atribut	Keterangan
1	gender	Jenis kelamin individu
2	age	Usia individu
3	hypertension	Riwayat hipertensi
4	heart_disease	Riwayat penyakit jantung
5	smoking_history	Riwayat merokok
6	bmi	Indeks massa tubuh
7	HbA1c_level	Tingkat HbA1c
8	blood_glucose_level	Kadar glukosa darah
9	diabetes	Status diabetes sebagai target klasifikasi

Import Library & Membaca Dataset

In [7]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report,
    mean_absolute_error, mean_squared_error, r2_score
)

FILE_PATH = "02_diabetes.csv"
df = pd.read_csv(FILE_PATH)

print("Ukuran dataset:", df.shape)
display(df.head())
#202431124_Abhillasitta
```

```
Ukuran dataset: (100000, 9)
```

Pada tahap awal penelitian, dilakukan proses persiapan lingkungan pemrograman dengan mengimpor beberapa library Python yang dibutuhkan untuk mendukung proses pengolahan data, analisis, visualisasi, serta pembangunan model Machine Learning. Library yang digunakan meliputi Pandas, NumPy, Matplotlib, dan Scikit-learn. Pandas digunakan untuk membaca dan mengelola dataset dalam bentuk tabel atau DataFrame, sedangkan NumPy digunakan untuk mendukung operasi numerik dan perhitungan matematis. Selanjutnya, Matplotlib digunakan untuk menghasilkan visualisasi data dalam bentuk grafik agar karakteristik dan pola data dapat diamati dengan lebih mudah. Sementara itu, Scikit-learn digunakan sebagai library utama dalam proses pengembangan model Machine Learning, yang mencakup pembagian dataset menggunakan train_test_split, penggabungan tahapan pengolahan data menggunakan Pipeline, serta penerapan transformasi atribut melalui ColumnTransformer, OneHotEncoder, dan StandardScaler. Selain itu, SimpleImputer digunakan untuk menangani kemungkinan adanya nilai kosong (missing values) pada dataset.

Dalam penelitian ini juga diimpor beberapa algoritma Machine Learning yang akan digunakan untuk membangun model prediksi. Algoritma tersebut meliputi DecisionTreeClassifier, KNeighborsClassifier, dan GaussianNB untuk menyelesaikan permasalahan klasifikasi, serta LinearRegression untuk menyelesaikan permasalahan regresi. Untuk mengukur performa model yang dihasilkan, digunakan beberapa metrik evaluasi, yaitu accuracy_score, precision_score, recall_score, f1_score, dan roc_auc_score untuk mengevaluasi hasil klasifikasi. Selain itu, confusion_matrix dan classification_report digunakan untuk memberikan gambaran lebih terperinci mengenai hasil prediksi model. Adapun evaluasi regresi dilakukan menggunakan metrik mean_absolute_error, mean_squared_error, dan r2_score untuk mengetahui tingkat kesalahan prediksi serta kemampuan model dalam menjelaskan variasi data. Dengan mengimpor seluruh library tersebut, program telah dipersiapkan untuk menjalankan tahapan penelitian mulai dari pengolahan dataset hingga evaluasi performa model.

Setelah seluruh library berhasil diimpor, langkah berikutnya adalah menentukan lokasi dataset yang akan digunakan melalui variabel FILE_PATH dengan nilai "02_diabetes.csv". Penentuan lokasi file ini bertujuan untuk mempermudah proses pemanggilan dataset ketika program dijalankan. Selanjutnya, perintah pd.read_csv(FILE_PATH) digunakan untuk membaca file CSV tersebut dan menyimpannya ke dalam variabel df dalam bentuk DataFrame. DataFrame

merupakan struktur data berbentuk tabel yang memungkinkan peneliti melakukan berbagai proses manipulasi, pemeriksaan, dan analisis terhadap data berdasarkan baris maupun kolom. Melalui proses ini, seluruh informasi yang terdapat dalam file dataset dapat diakses dan diolah menggunakan berbagai fungsi yang tersedia pada library Pandas.

Untuk memastikan bahwa dataset berhasil dibaca dengan benar, digunakan perintah print("Ukuran dataset:", df.shape). Atribut shape berfungsi untuk menampilkan dimensi dataset yang terdiri atas jumlah baris dan jumlah kolom. Berdasarkan hasil eksekusi program, diperoleh ukuran dataset sebesar (100000, 9), yang menunjukkan bahwa dataset memiliki sebanyak 100.000 baris data dan 9 kolom atribut. Hasil tersebut sesuai dengan struktur dataset yang digunakan dalam penelitian ini. Selanjutnya, perintah display(df.head()) digunakan untuk menampilkan lima baris pertama dari dataset. Penggunaan fungsi head() bertujuan untuk memberikan gambaran awal mengenai isi dataset, meliputi nama atribut, jenis informasi, serta contoh nilai yang terdapat pada masing-masing kolom. Melalui tampilan tersebut, peneliti dapat melakukan pemeriksaan awal terhadap kesesuaian struktur data sebelum melanjutkan ke tahapan pengolahan dan analisis lebih lanjut.

Abhilla_202431124

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_glucose_level	diabetes
0	Female	80.0	0	1	never	25.19	6.6	140	0
1	Female	54.0	0	0	No Info	27.32	6.6	80	0
2	Male	28.0	0	0	never	27.32	5.7	158	0
3	Female	36.0	0	0	current	23.45	5.0	155	0
4	Male	76.0	1	1	current	20.14	4.8	155	0

Berdasarkan hasil eksekusi program, diperoleh ukuran dataset sebesar (100000, 9), yang menunjukkan bahwa dataset memiliki 100.000 baris data dan 9 kolom atribut. Selanjutnya, ditampilkan lima baris pertama menggunakan fungsi df.head() untuk memberikan gambaran awal mengenai isi dataset. Berdasarkan hasil tersebut, dapat diketahui bahwa data mencakup informasi jenis kelamin (gender), usia (age), riwayat hipertensi (hypertension), riwayat penyakit jantung (heart_disease), riwayat merokok (smoking_history), indeks massa tubuh (bmi), tingkat HbA1c (HbA1c_level), kadar glukosa darah (blood_glucose_level), serta status diabetes (diabetes). Nilai pada atribut hypertension, heart_disease, dan diabetes direpresentasikan menggunakan angka biner, yaitu 0 dan 1. Sementara itu, atribut kategorikal seperti gender dan smoking_history masih menggunakan nilai berbentuk teks. Hasil pemeriksaan awal ini menunjukkan bahwa dataset telah berhasil dimuat

dan memiliki struktur atribut yang sesuai dengan kebutuhan penelitian. Data tersebut selanjutnya akan melalui tahap pemeriksaan dan pengolahan sebelum digunakan dalam proses pembangunan model Machine Learning.

3.2 Exploratory Data Analysis (EDA)

Tahap Exploratory Data Analysis (EDA) dilakukan untuk memahami karakteristik, struktur, distribusi, serta hubungan antaratribut dalam dataset sebelum dilakukan proses pemodelan. Pada tahap ini, dilakukan pemeriksaan terhadap tipe data, statistik deskriptif, keberadaan nilai kosong (missing values), data duplikat, serta distribusi variabel target. Proses ini bertujuan untuk mengetahui kondisi dan kualitas dataset sehingga dapat menentukan langkah pengolahan data yang sesuai sebelum diterapkan algoritma Machine Learning.

EDA

In [8]:
```
# Struktur, tipe data, dan statistik deskriptif
df.info()
display(df.describe(include="all").T)

# Missing values dan duplikasi
print("Missing values per kolom:")
display(df.isnull().sum().to_frame("Jumlah Missing Values"))

print("Jumlah baris duplikat:", df.duplicated().sum())

# Distribusi target
print("Distribusi target:")
display(df["diabetes"].value_counts().rename(index={0:"Tidak diabetes", 1:"Diabetes"}))
display((df["diabetes"].value_counts(normalize=True)*100).round(2))
#202431124_Abhillasitta
```

Kode program diawali dengan perintah df.info() yang digunakan untuk menampilkan informasi struktur dataset, meliputi jumlah data, nama kolom, jumlah nilai yang tidak kosong (non-null values), serta tipe data pada setiap atribut. Selanjutnya, perintah df.describe(include="all").T digunakan untuk menghasilkan statistik deskriptif dari seluruh atribut, baik numerik maupun kategorikal. Statistik tersebut mencakup jumlah data, nilai rata-rata, standar deviasi, nilai minimum, maksimum, serta informasi frekuensi dan kategori yang terdapat dalam dataset. Penggunaan .T bertujuan untuk mengubah orientasi tabel agar setiap atribut ditampilkan dalam baris sehingga lebih mudah dibaca dan dianalisis.

Tahap berikutnya adalah pemeriksaan nilai kosong dan data duplikat. Perintah df.isnull().sum() digunakan untuk menghitung jumlah nilai kosong pada setiap kolom, kemudian to_frame("Jumlah Missing Values") digunakan untuk menyajikan hasilnya dalam bentuk tabel. Pemeriksaan ini penting untuk mengetahui apakah terdapat data yang belum terisi dan memerlukan penanganan sebelum proses pemodelan. Sementara itu, perintah df.duplicated().sum() digunakan untuk menghitung jumlah baris yang memiliki data identik atau duplikat. Pemeriksaan

tersebut bertujuan untuk memastikan kualitas dataset serta mengidentifikasi kemungkinan adanya pengulangan data yang dapat memengaruhi hasil analisis.

Selanjutnya, dilakukan pemeriksaan distribusi variabel target menggunakan atribut diabetes. Perintah df["diabetes"].value_counts() digunakan untuk menghitung jumlah data pada setiap kelas, yaitu nilai 0 yang menunjukkan individu tidak menderita diabetes dan nilai 1 yang menunjukkan individu menderita diabetes. Penggunaan rename() bertujuan untuk mengubah representasi angka tersebut menjadi keterangan yang lebih mudah dipahami. Kemudian, value_counts(normalize=True)*100 digunakan untuk menghitung persentase masing-masing kelas dalam dataset, sedangkan round(2) berfungsi membulatkan hasil persentase hingga dua angka di belakang koma. Hasil pemeriksaan distribusi ini digunakan untuk mengetahui keseimbangan jumlah data antara kelas diabetes dan tidak diabetes, sehingga dapat menjadi pertimbangan dalam menentukan metode evaluasi dan strategi pemodelan yang tepat.

```
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   gender               100000 non-null  object
 1   age                  100000 non-null  float64
 2   hypertension         100000 non-null  int64
 3   heart_disease        100000 non-null  int64
 4   smoking_history      100000 non-null  object
 5   bmi                  100000 non-null  float64
 6   HbA1c_level          100000 non-null  float64
 7   blood_glucose_level  100000 non-null  int64
 8   diabetes             100000 non-null  int64
dtypes: float64(3), int64(4), object(2)
memory usage: 6.9+ MB
```

Abhilla_202431124

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
gender	100000	3	Female	58552	NaN	NaN	NaN	NaN	NaN	NaN	NaN
age	100000.0	NaN	NaN	NaN	41.885856	22.51684	0.08	24.0	43.0	60.0	80.0
hypertension	100000.0	NaN	NaN	NaN	0.07485	0.26315	0.0	0.0	0.0	0.0	1.0
heart_disease	100000.0	NaN	NaN	NaN	0.03942	0.194593	0.0	0.0	0.0	0.0	1.0
smoking_history	100000	6	No Info	35816	NaN	NaN	NaN	NaN	NaN	NaN	NaN
bmi	100000.0	NaN	NaN	NaN	27.320767	6.636783	10.01	23.63	27.32	29.58	95.69
HbA1c_level	100000.0	NaN	NaN	NaN	5.527507	1.070672	3.5	4.8	5.8	6.2	9.0
blood_glucose_level	100000.0	NaN	NaN	NaN	138.05806	40.708136	80.0	100.0	140.0	159.0	300.0
diabetes	100000.0	NaN	NaN	NaN	0.085	0.278883	0.0	0.0	0.0	0.0	1.0

```
Missing values per kolom:
```

	Jumlah Missing Values
gender	0
age	0
hypertension	0
heart_disease	0
smoking_history	0
bmi	0
HbA1c_level	0
blood_glucose_level	0
diabetes	0

Abhilla_202431124

```
Jumlah baris duplikat: 3854
Distribusi target:

Tidak diabetes    91500
Diabetes           8500
Name: diabetes, dtype: int64

0    91.5
1     8.5
Name: diabetes, dtype: float64
```

Berdasarkan hasil eksekusi program EDA, diketahui bahwa dataset memiliki 100.000 baris data dan 9 kolom atribut dengan penggunaan memori sebesar 6,9 MB. Hasil pemeriksaan tipe data menunjukkan bahwa terdapat tiga atribut bertipe float64, empat atribut bertipe int64, dan dua atribut bertipe object. Atribut kategorikal yang bertipe object adalah gender dan smoking_history, sedangkan atribut lainnya merupakan data numerik. Berdasarkan statistik deskriptif, diperoleh rata-rata usia individu sebesar 41,89 tahun, rata-rata BMI sebesar 27,32, tingkat HbA1c sebesar 5,53, serta kadar glukosa darah sebesar 138,06. Selain itu, atribut gender memiliki tiga kategori berbeda, sedangkan smoking_history memiliki enam kategori. Hasil tersebut memberikan gambaran mengenai karakteristik umum individu yang terdapat dalam dataset penelitian.

Pada pemeriksaan missing values, seluruh sembilan atribut menunjukkan jumlah nilai kosong sebanyak 0. Hal ini menandakan bahwa dataset tidak memiliki data yang hilang sehingga tidak diperlukan penanganan khusus terhadap nilai kosong. Namun, berdasarkan pemeriksaan duplikasi menggunakan df.duplicated().sum(), ditemukan sebanyak 3.854 baris data duplikat. Temuan ini perlu diperhatikan dalam tahap preprocessing karena data yang berulang berpotensi memengaruhi proses pembelajaran dan hasil evaluasi model. Oleh karena itu, data duplikat perlu dipertimbangkan untuk dihapus agar proses pemodelan menggunakan data yang lebih bersih.

Selanjutnya, hasil pemeriksaan distribusi variabel target diabetes menunjukkan bahwa terdapat 91.500 data atau sebesar 91,5% individu yang tidak menderita diabetes, sedangkan 8.500 data atau sebesar 8,5% individu menderita diabetes. Hasil tersebut memperlihatkan adanya ketidakseimbangan jumlah data (class imbalance) antara kedua kelas, dengan jumlah data individu tanpa diabetes jauh lebih besar dibandingkan individu yang menderita diabetes. Kondisi ini perlu diperhatikan dalam pembangunan model klasifikasi karena dapat menyebabkan model cenderung lebih dominan memprediksi kelas mayoritas. Oleh karena itu, evaluasi model nantinya tidak hanya berfokus pada nilai akurasi, tetapi juga mempertimbangkan nilai precision, recall, dan F1-score agar kemampuan model dalam mengenali kedua kelas dapat dinilai secara lebih menyeluruh.

3.2.1 Analisis Distribusi Target Diabetes

Analisis distribusi target dilakukan untuk mengetahui perbandingan jumlah individu yang termasuk dalam kategori diabetes dan tidak diabetes.

In [9]:
```
# Visualisasi distribusi target
target_counts = df["diabetes"].value_counts().sort_index()
plt.figure(figsize=(6,4))
plt.bar(["Tidak diabetes (0)", "Diabetes (1)"], target_counts.values)
plt.title("Distribusi Status Diabetes")
plt.ylabel("Jumlah Data")
plt.tight_layout()
plt.show()

# Histogram fitur numerik
numeric_cols = df.select_dtypes(include=np.number).columns.tolist()
df[numeric_cols].hist(figsize=(12,9), bins=25)
plt.suptitle("Distribusi Fitur Numerik")
plt.tight_layout()
plt.show()
#202431124_Abhillasitta
```

Analisis distribusi target dilakukan untuk mengetahui perbandingan jumlah individu yang termasuk dalam kategori diabetes dan tidak diabetes. Tahap ini bertujuan untuk mengidentifikasi pola persebaran kelas pada atribut diabetes serta mengetahui apakah terdapat ketidakseimbangan jumlah data antara kedua kategori tersebut. Proses visualisasi pertama dilakukan dengan menggunakan perintah df["diabetes"].value_counts().sort_index() untuk menghitung jumlah data pada setiap kategori target dan mengurutkannya berdasarkan nilai indeks. Selanjutnya, plt.figure(figsize=(6,4)) digunakan untuk menentukan ukuran grafik, sedangkan plt.bar() digunakan untuk membentuk diagram batang yang menampilkan perbandingan jumlah individu yang tidak menderita diabetes (0) dan individu yang menderita diabetes (1). Perintah plt.title() digunakan untuk memberikan judul grafik, plt.ylabel() untuk memberikan keterangan pada sumbu vertikal, serta plt.tight_layout() dan plt.show() untuk mengatur tata letak dan menampilkan hasil visualisasi.

Selain visualisasi distribusi target, dilakukan pula analisis persebaran atribut numerik menggunakan histogram. Proses ini diawali dengan perintah df.select_dtypes(include=np.number).columns.tolist()untuk mengidentifikasi seluruh kolom yang memiliki tipe data numerik. Selanjutnya, df[numeric_cols].hist(figsize=(12,9), bins=25) digunakan untuk menghasilkan histogram dari setiap atribut numerik dengan ukuran visualisasi 12 × 9 inci dan pembagian sebanyak 25 interval (bins). Perintah plt.suptitle() digunakan untuk memberikan judul utama berupa "Distribusi Fitur Numerik", sedangkan plt.tight_layout() dan plt.show() digunakan untuk mengatur tampilan serta menampilkan seluruh histogram.



a. Distribusi Status Diabetes (Bar Chart): Grafik batang tunggal di bagian atas menunjukkan perbandingan jumlah individu berdasarkan variabel target. Visualisasi ini memperlihatkan bahwa dataset sangat tidak seimbang (*imbalanced*), di mana mayoritas data masuk dalam kategori "Tidak diabetes (0)" (mendekati 90.000), sedangkan kategori "Diabetes (1)" jumlahnya jauh lebih sedikit (di bawah 10.000).

b. Distribusi Fitur Numerik (Histogram): Kumpulan grafik di bagian bawah (subplot) menampilkan sebaran frekuensi untuk semua kolom yang bertipe numerik.

- Kolom yang bersifat biner (hanya memiliki nilai 0 dan 1) seperti hypertension, heart_disease, dan diabetes sangat didominasi oleh nilai 0.
- Kolom age (usia) menunjukkan distribusi data yang cukup merata di berbagai kelompok usia, namun terdapat lonjakan frekuensi yang tajam pada rentang usia sekitar 80 tahun.

- Kolom bmi memiliki distribusi yang sedikit condong ke kanan (*right-skewed*), dengan data paling banyak terkumpul pada rentang nilai 25 hingga 30.
- Kolom HbA1c_level dan blood_glucose_level menunjukkan pola distribusi dengan beberapa puncak atau kelompok nilai tertentu (*discrete peaks*), menandakan mayoritas data berkumpul pada angka-angka spesifik seperti kadar glukosa di sekitar 150 atau HbA1c di sekitar 6.0.

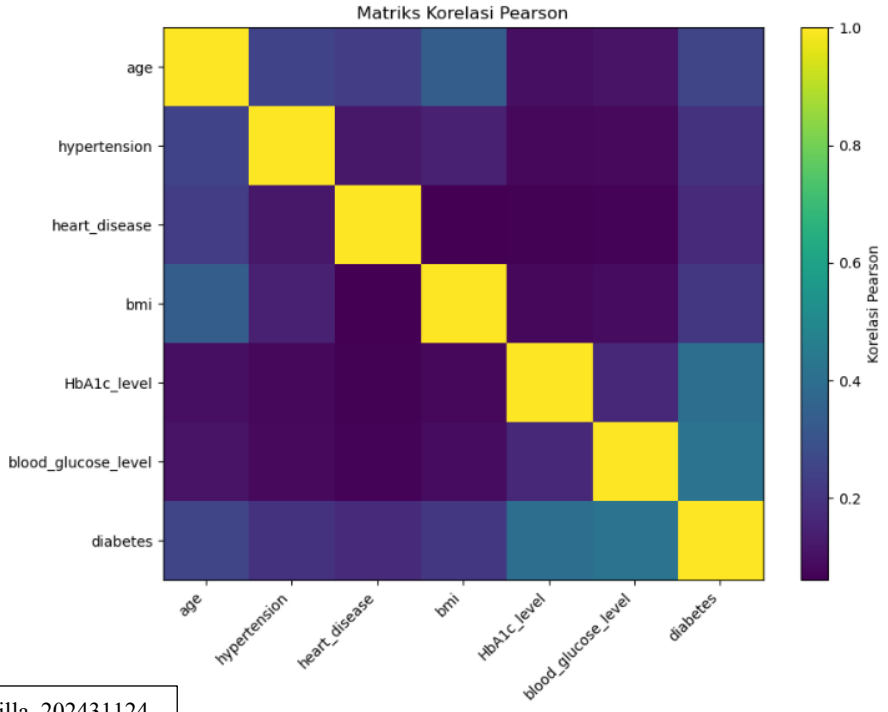
3.2.2 Analisis Korelasi Antaratribut

Analisis korelasi dilakukan menggunakan metode Pearson untuk mengetahui hubungan linear antaratribut numerik.

```
In [10]:
# Korelasi Pearson antarfitur numerik
corr = df[numeric_cols].corr(method="pearson")
plt.figure(figsize=(9,7))
plt.imshow(corr, aspect="auto")
plt.colorbar(label="Korelasi Pearson")
plt.xticks(range(len(corr.columns)), corr.columns, rotation=45, ha="right")
plt.yticks(range(len(corr.index)), corr.index)
plt.title("Matriks Korelasi Pearson")
plt.tight_layout()
plt.show()
display(corr.round(3))
#202431124_Abhillasitta
```

Tahap ini bertujuan untuk mengidentifikasi tingkat kekuatan dan arah hubungan antara satu atribut dengan atribut lainnya, termasuk hubungannya terhadap variabel target diabetes. Proses analisis diawali dengan perintah df[numeric_cols].corr(method="pearson") yang digunakan untuk menghitung nilai korelasi Pearson pada seluruh atribut numerik. Nilai korelasi yang dihasilkan berada pada rentang -1 hingga +1, di mana nilai positif menunjukkan hubungan searah, nilai negatif menunjukkan hubungan berlawanan arah, sedangkan nilai yang mendekati 0 menunjukkan hubungan linear yang lemah atau tidak terlihat. Selanjutnya, digunakan plt.figure(figsize=(9,7)) untuk menentukan ukuran visualisasi dan plt.imshow(corr, aspect="auto") untuk menampilkan hasil korelasi dalam bentuk matriks berwarna (heatmap). Perintah plt.colorbar() digunakan untuk menampilkan skala warna berdasarkan nilai korelasi, sedangkan plt.xticks() dan plt.yticks() digunakan untuk memberikan label nama atribut pada sumbu horizontal dan vertikal. Kemudian, plt.title() memberikan judul visualisasi, sementara plt.tight_layout() dan plt.show() digunakan untuk mengatur tata letak serta menampilkan grafik. Setelah visualisasi ditampilkan, perintah

display(corr.round(3)) digunakan untuk menyajikan hasil perhitungan korelasi dalam bentuk tabel dengan pembulatan hingga tiga angka di belakang koma.



Abhilla_202431124

	age	hypertension	heart_disease	bmi	HbA1c_level	blood_glucose_level	diabetes
age	1.000	0.251	0.233	0.337	0.101	0.111	0.258
hypertension	0.251	1.000	0.121	0.148	0.081	0.084	0.198
heart_disease	0.233	0.121	1.000	0.061	0.068	0.070	0.172
bmi	0.337	0.148	0.061	1.000	0.083	0.091	0.214
HbA1c_level	0.101	0.081	0.068	0.083	1.000	0.167	0.401
blood_glucose_level	0.111	0.084	0.070	0.091	0.167	1.000	0.420
diabetes	0.258	0.198	0.172	0.214	0.401	0.420	1.000

Analisis Matriks Korelasi Pearson untuk mengukur tingkat hubungan linear antara berbagai fitur numerik dalam dataset. Hasilnya ditampilkan dalam dua bentuk: sebuah heatmap (peta panas) bergradasi warna dan sebuah tabel yang memuat nilai korelasi spesifik yang telah dibulatkan menjadi tiga angka di belakang koma. Pada visualisasi heatmap, kotak-kotak yang membentuk garis diagonal berwarna kuning terang memiliki nilai sempurna 1.0 karena mewakili korelasi suatu variabel dengan dirinya sendiri.

Berdasarkan nilai pada tabel dan intensitas warna pada heatmap, dapat diamati bahwa fitur blood_glucose_level dan HbA1c_level memiliki hubungan korelasi positif yang paling kuat terhadap variabel target diabetes, dengan nilai masing-masing sebesar 0.420 dan 0.401. Selain itu, fitur age (usia) dan bmi juga menunjukkan korelasi yang cukup berarti terhadap diabetes yakni 0.258 dan 0.214. Untuk hubungan antar sesama variabel independen (bukan target), korelasi paling tinggi terjadi antara age dan bmi

di angka 0.337, sedangkan sisa area yang didominasi warna ungu gelap kebiruan mengindikasikan bahwa korelasi antar fitur lainnya tergolong rendah.

3.3 Exploratory Data Analysis (EDA)

Tahap preprocessing dilakukan untuk mempersiapkan dataset agar dapat digunakan oleh algoritma Machine Learning. Proses ini meliputi penghapusan data duplikat, pemisahan fitur dan target, identifikasi tipe data, serta pembagian data pelatihan dan pengujian.

Preprocessing Data

In [11]:
```
# Hapus baris duplikat
data = df.drop_duplicates().copy()
print("Ukuran sebelum menghapus duplikat:", df.shape)
print("Ukuran setelah menghapus duplikat:", data.shape)
print("Duplikat tersisa:", data.duplicated().sum())

# Pisahkan fitur dan target klasifikasi
X = data.drop(columns="diabetes")
y = data["diabetes"]

# Identifikasi kolom kategorikal dan numerik
cat_cols = X.select_dtypes(include="object").columns.tolist()
num_cols = X.select_dtypes(exclude="object").columns.tolist()

print("Kolom kategorikal:", cat_cols)
print("Kolom numerik:", num_cols)

# Split stratified: 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print("Data latih:", X_train.shape)
print("Data uji:", X_test.shape)
print("Proporsi target data uji:")
display(y_test.value_counts(normalize=True).round(3))
#202431124_Abhillasitta
```

Pada tahap ini, langkah pertama yang dilakukan adalah menghapus baris data duplikat menggunakan perintah df.drop_duplicates().copy() dan menyimpan hasilnya ke dalam variabel data. Proses ini bertujuan untuk menghilangkan data yang memiliki isi identik agar tidak terjadi pengulangan informasi yang berpotensi memengaruhi proses pembelajaran model. Selanjutnya, digunakan perintah df.shape dan data.shape untuk membandingkan ukuran dataset sebelum dan sesudah penghapusan duplikasi, sedangkan data.duplicated().sum() digunakan untuk memastikan bahwa tidak terdapat lagi baris duplikat dalam dataset.

Setelah data duplikat dihapus, dilakukan pemisahan antara fitur (features) dan target klasifikasi (target) menggunakan perintah X = data.drop(columns="diabetes") dan y = data["diabetes"]. Variabel X berisi seluruh atribut yang digunakan sebagai variabel prediktor, yaitu gender, usia, hipertensi, penyakit jantung, riwayat merokok, BMI, tingkat HbA1c, dan kadar glukosa darah. Sementara itu, variabel y berisi atribut diabetes yang berfungsi sebagai variabel target untuk memprediksi status diabetes individu. Selanjutnya, dilakukan identifikasi jenis atribut menggunakan

select_dtypes() untuk memisahkan kolom kategorikal dan numerik. Atribut kategorikal disimpan dalam variabel cat_cols, sedangkan atribut numerik disimpan dalam variabel num_cols. Pemisahan ini diperlukan karena kedua jenis data tersebut membutuhkan metode pengolahan yang berbeda pada tahap transformasi data berikutnya.

Tahap selanjutnya adalah membagi dataset menjadi data pelatihan (training data) dan data pengujian (testing data) menggunakan fungsi train_test_split(). Pembagian dilakukan dengan proporsi 80% untuk data pelatihan dan 20% untuk data pengujian melalui parameter test_size=0.20. Parameter random_state=42 digunakan untuk memastikan pembagian data menghasilkan pola yang konsisten ketika program dijalankan kembali, sedangkan stratify=y digunakan untuk mempertahankan proporsi kelas diabetes dan tidak diabetes pada kedua kelompok data. Penggunaan metode stratified splitting sangat penting karena dataset memiliki distribusi kelas yang tidak seimbang, sehingga proporsi masing-masing kelas tetap terjaga pada data pelatihan maupun pengujian. Setelah proses pembagian selesai, perintah X_train.shape dan X_test.shape digunakan untuk mengetahui jumlah data pada masing-masing kelompok, sedangkan y_test.value_counts(normalize=True).round(3) digunakan untuk memeriksa proporsi distribusi target pada data pengujian. Dengan demikian, data telah terbagi menjadi dua bagian yang akan digunakan untuk proses pelatihan model dan evaluasi performa model secara terpisah.

```
Ukuran sebelum menghapus duplikat: (100000, 9)
Ukuran setelah menghapus duplikat: (96146, 9)
Duplikat tersisa: 0
Kolom kategorikal: ['gender', 'smoking_history']
Kolom numerik: ['age', 'hypertension', 'heart_disease', 'bmi', 'HbA1c_level', 'blood_glucose_level']
Data latih: (76916, 8)
Data uji: (19230, 8)
Proporsi target data uji:

0    0.912
1    0.088
Name: diabetes, dtype: float64
```

Abhilla_202431124

Berdasarkan hasil eksekusi program data preprocessing dan pembagian dataset, diperoleh beberapa hasil sebagai berikut:

1. Penghapusan Data Duplikat

   Hasil menunjukkan bahwa ukuran dataset sebelum penghapusan duplikasi adalah 100.000 baris dan 9 kolom. Setelah dilakukan proses penghapusan menggunakan drop_duplicates(), jumlah data berkurang menjadi 96.146 baris dan 9 kolom. Dengan demikian, sebanyak 3.854 baris data duplikat berhasil dihapus dan tidak ditemukan lagi data duplikat setelah proses tersebut dilakukan.

2. Identifikasi Atribut Kategorikal dan Numerik

   Berdasarkan hasil identifikasi tipe data, diperoleh dua atribut kategorikal, yaitu gender dan smoking_history. Sementara itu, terdapat enam atribut numerik, yaitu age, hypertension, heart_disease, bmi, HbA1c_level, dan blood_glucose_level. Pemisahan ini dilakukan untuk mempermudah penerapan metode transformasi data yang sesuai dengan karakteristik masing-masing atribut pada tahap preprocessing berikutnya.

3. Pembagian Data Pelatihan dan Pengujian

   Dataset yang telah dibersihkan kemudian dibagi menggunakan metode train-test split dengan proporsi 80% data pelatihan dan 20% data pengujian. Hasil pembagian menunjukkan bahwa data pelatihan memiliki 76.916 baris dengan 8 atribut, sedangkan data pengujian memiliki 19.230 baris dengan 8 atribut. Atribut target diabetes telah dipisahkan sehingga tidak termasuk dalam jumlah kolom fitur tersebut.

4. Pemeriksaan Proporsi Target Data Uji

   Hasil pemeriksaan distribusi target pada data pengujian menunjukkan bahwa sebanyak 91,2% data termasuk kategori tidak diabetes (0), sedangkan 8,8% lainnya termasuk kategori diabetes (1). Proporsi tersebut relatif mendekati distribusi kelas pada dataset sebelum pembagian. Hal ini menunjukkan bahwa penggunaan parameter stratify=y berhasil mempertahankan proporsi masing-masing kelas pada proses pembagian data.

3.4 Pelatihan dan Evaluasi Model Klasifikasi

Pada penelitian ini, terdapat tiga algoritma klasifikasi yang digunakan, yaitu Decision Tree, K-Nearest Neighbors (KNN), dan Gaussian Naive Bayes. Ketiga algoritma dilatih menggunakan data pelatihan yang sama dan dievaluasi menggunakan data pengujian yang sama. Hal ini dilakukan agar hasil perbandingan dapat memberikan gambaran mengenai kemampuan masing-masing algoritma pada dataset yang digunakan. Evaluasi dilakukan menggunakan lima metrik, yaitu Accuracy, Precision, Recall, F1-Score, dan ROC-AUC.

Pelatihan dan evaluasi model klasifikasi

In [12]:
```
def make_preprocessor(scale_numeric=False):
    numeric_steps = [
        ("imputer", SimpleImputer(strategy="median"))
    ]
    if scale_numeric:
        numeric_steps.append(
            ("scaler", StandardScaler())
        )
    numeric_pipeline = Pipeline(numeric_steps)
    categorical_pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(
            handle_unknown="ignore",
            sparse=False
        ))
    ])
    return ColumnTransformer([
        ("num", numeric_pipeline, num_cols),
        ("cat", categorical_pipeline, cat_cols)
    ])
models = {
    "Decision Tree": Pipeline([
        ("preprocessing", make_preprocessor(False)),
        ("model", DecisionTreeClassifier(random_state=42))
    ]),
    "KNN (k=5)": Pipeline([
        ("preprocessing", make_preprocessor(True)),
        ("model", KNeighborsClassifier(n_neighbors=5))
    ]),
    "Gaussian Naive Bayes": Pipeline([
        ("preprocessing", make_preprocessor(True)),
        ("model", GaussianNB())
    ])
}
```

```
results = []
fitted_models = {}
predictions = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_probability = model.predict_proba(X_test)[:, 1]
    fitted_models[name] = model
    predictions[name] = y_pred
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(
            y_test, y_pred, zero_division=0
        ),
        "Recall": recall_score(
            y_test, y_pred, zero_division=0
        ),
        "F1-Score": f1_score(
            y_test, y_pred, zero_division=0
        ),
        "ROC-AUC": roc_auc_score(
            y_test, y_probability
        )
    })
results_df = pd.DataFrame(results).set_index("Model")
display((results_df * 100).round(2))
#202431124_Abhillasitta
```

Pada bagian awal, dibuat sebuah fungsi bernama make_preprocessor(scale_numeric=False) yang bertujuan untuk membangun tahapan preprocessing secara terstruktur. Di dalam fungsi tersebut, atribut numerik diproses menggunakan SimpleImputer(strategy="median") untuk menangani kemungkinan nilai kosong dengan menggantinya menggunakan nilai median.

Apabila parameter scale_numeric bernilai True, program juga menerapkan StandardScaler() untuk melakukan standarisasi terhadap atribut numerik sehingga memiliki skala yang lebih seragam. Sementara itu, atribut kategorikal diproses menggunakan SimpleImputer(strategy="most_frequent") untuk mengganti nilai kosong berdasarkan kategori yang paling sering muncul, kemudian dilanjutkan dengan OneHotEncoder(handle_unknown="ignore", sparse=False) untuk mengubah data kategorikal menjadi representasi numerik. Parameter handle_unknown="ignore" digunakan agar program tetap dapat memproses kategori yang belum pernah ditemukan saat pelatihan, sedangkan sparse=False digunakan agar hasil transformasi disimpan dalam bentuk array biasa. Seluruh tahapan tersebut kemudian digabungkan menggunakan ColumnTransformer, sehingga preprocessing dapat diterapkan secara otomatis sesuai dengan kelompok atribut numerik dan kategorikal.

Selanjutnya, dibuat variabel models yang berisi tiga algoritma klasifikasi dalam bentuk Pipeline. Model pertama adalah Decision Tree yang menggunakan preprocessing tanpa standarisasi numerik melalui parameter make_preprocessor(False), kemudian dilanjutkan dengan algoritma DecisionTreeClassifier(random_state=42). Model kedua adalah KNN dengan jumlah tetangga sebanyak lima melalui parameter n_neighbors=5, yang menggunakan preprocessing dengan standarisasi numerik. Model ketiga adalah Gaussian Naive Bayes yang menggunakan GaussianNB() dan juga menerapkan standarisasi numerik. Penggunaan preprocessing yang berbeda disesuaikan dengan karakteristik masing-masing algoritma, terutama karena KNN menggunakan perhitungan jarak sehingga membutuhkan skala atribut yang seragam. Sementara itu, Decision Tree tidak bergantung pada perhitungan jarak sehingga standarisasi numerik tidak menjadi kebutuhan utama.

Setelah seluruh model didefinisikan, proses pelatihan dan evaluasi dilakukan menggunakan perulangan for name, model in models.items(). Pada setiap iterasi, perintah model.fit(X_train, y_train) digunakan untuk melatih model berdasarkan data pelatihan. Selanjutnya, model.predict(X_test) digunakan untuk menghasilkan prediksi kelas diabetes pada data pengujian, sedangkan model.predict_proba(X_test)[:, 1] digunakan untuk memperoleh probabilitas prediksi terhadap kelas diabetes (kelas 1). Model yang telah dilatih disimpan ke dalam variabel fitted_models, sementara hasil prediksi masing-masing algoritma disimpan dalam variabel predictions. Penyimpanan ini bertujuan agar model dan

hasil prediksinya dapat digunakan kembali pada tahapan evaluasi maupun analisis berikutnya.

Evaluasi performa masing-masing algoritma dilakukan menggunakan lima metrik, yaitu Accuracy, Precision, Recall, F1-Score, dan ROC-AUC. Accuracy digunakan untuk mengukur persentase keseluruhan prediksi yang benar, Precision mengukur ketepatan model ketika memprediksi individu yang menderita diabetes, Recall mengukur kemampuan model dalam menemukan individu yang benar-benar menderita diabetes, sedangkan F1-Score merupakan nilai keseimbangan antara Precision dan Recall. Adapun ROC-AUC digunakan untuk mengukur kemampuan model dalam membedakan kelas diabetes dan tidak diabetes berdasarkan probabilitas prediksinya. Seluruh hasil evaluasi kemudian disimpan ke dalam variabel results dalam bentuk dictionary untuk setiap algoritma.

Pada tahap akhir, hasil evaluasi dikonversi menjadi DataFrame menggunakan pd.DataFrame(results).set_index("Model") agar dapat disajikan dalam bentuk tabel berdasarkan nama algoritma. Perintah (results_df * 100).round(2) digunakan untuk mengubah nilai metrik menjadi persentase dan membulatkannya hingga dua angka di belakang koma. Dengan demikian, hasil evaluasi ketiga algoritma dapat ditampilkan secara berdampingan sehingga mempermudah proses perbandingan performa dan penentuan algoritma klasifikasi yang memberikan hasil terbaik pada dataset diabetes.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Decision Tree	94.78	69.01	74.17	71.50	85.57
KNN (k=5)	95.93	88.52	61.85	72.82	90.08
Gaussian Naive Bayes	62.28	18.49	96.17	31.02	90.65

Abhilla_202431124

Berdasarkan hasil evaluasi terhadap ketiga algoritma klasifikasi, diperoleh nilai performa yang berbeda pada setiap metrik pengujian. Hasil tersebut menunjukkan bahwa masing-masing algoritma memiliki kemampuan yang berbeda dalam memprediksi status diabetes. Adapun interpretasi hasil evaluasi tersebut adalah sebagai berikut:

1. Decision Tree

   Algoritma Decision Tree memperoleh nilai Accuracy sebesar 94,78%, Precision sebesar 69,01%, Recall sebesar 74,17%, F1-Score sebesar 71,50%, dan ROC-AUC sebesar 85,57%. Hasil ini menunjukkan bahwa model memiliki

kemampuan prediksi keseluruhan yang cukup baik. Nilai Recall sebesar 74,17% menunjukkan bahwa model mampu mengenali sebagian besar individu yang benar-benar menderita diabetes. Sementara itu, nilai Precision sebesar 69,01% menunjukkan bahwa masih terdapat sejumlah prediksi positif yang tidak sesuai dengan kondisi sebenarnya.

2. K-Nearest Neighbors (KNN)

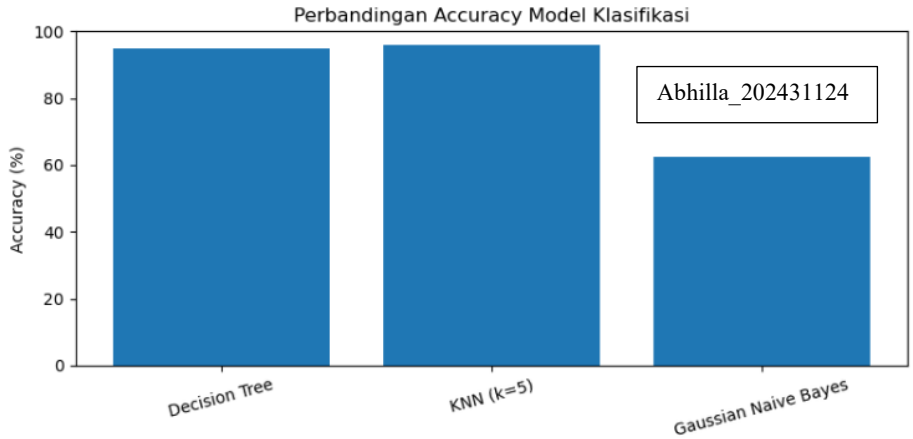
   Algoritma KNN dengan jumlah tetangga sebanyak lima memperoleh Accuracy sebesar 95,93%, Precision sebesar 88,52%, Recall sebesar 61,85%, F1-Score sebesar 72,82%, dan ROC-AUC sebesar 90,08%. Berdasarkan hasil tersebut, KNN menghasilkan nilai Accuracy, Precision, dan ROC-AUC yang lebih tinggi dibandingkan kedua algoritma lainnya. Hal ini menunjukkan bahwa KNN memiliki kemampuan yang baik dalam menghasilkan prediksi positif yang tepat serta membedakan individu yang menderita diabetes dan tidak diabetes. Namun, nilai Recall sebesar 61,85% menunjukkan bahwa model masih belum mampu mendeteksi seluruh individu yang benar-benar menderita diabetes secara optimal.

3. Gaussian Naive Bayes

   Algoritma Gaussian Naive Bayes memperoleh Accuracy sebesar 62,28%, Precision sebesar 18,49%, Recall sebesar 96,17%, F1-Score sebesar 31,02%, dan ROC-AUC sebesar 90,65%. Hasil tersebut menunjukkan bahwa model memiliki nilai Recall tertinggi dibandingkan kedua algoritma lainnya, yaitu sebesar 96,17%. Artinya, model mampu mendeteksi sebagian besar individu yang benar-benar menderita diabetes. Namun, nilai Precision dan Accuracy yang relatif rendah menunjukkan bahwa model juga menghasilkan banyak prediksi positif yang sebenarnya termasuk kategori tidak diabetes. Meskipun demikian, Gaussian Naive Bayes memiliki nilai ROC-AUC tertinggi, yaitu 90,65%, yang menunjukkan kemampuan diskriminasi kelas yang baik berdasarkan probabilitas prediksi.

In [16]:
```
# Grafik perbandingan accuracy
plt.figure(figsize=(8,4))
plt.bar(results_df.index, results_df["Accuracy"] * 100)
plt.ylabel("Accuracy (%)")
plt.title("Perbandingan Accuracy Model Klasifikasi")
plt.ylim(0, 100)
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
#202431124_Abhillasitta
```

Grafik perbandingan nilai Accuracy dari tiga model klasifikasi, yaitu Decision Tree, K-Nearest Neighbors (KNN), dan Gaussian Naive Bayes. Grafik dibuat menggunakan bar chart atau diagram batang untuk mempermudah pengamatan terhadap perbedaan tingkat akurasi setiap model. Fungsi plt.figure(figsize=(8,4)) digunakan untuk mengatur ukuran grafik, sedangkan plt.bar(results_df.index, results_df["Accuracy"] * 100) digunakan untuk menampilkan nama model pada sumbu X dan nilai Accuracy dalam bentuk persentase pada sumbu Y. Selanjutnya, plt.ylabel() memberikan keterangan pada sumbu Y, plt.title() menentukan judul grafik, dan plt.ylim(0,100) mengatur rentang nilai akurasi dari 0 hingga 100%. Pengaturan plt.xticks(rotation=15) berfungsi memiringkan label nama model agar lebih mudah dibaca, sementara plt.tight_layout() merapikan tata letak grafik dan plt.show() menampilkan hasil visualisasi.



Berdasarkan hasil evaluasi sebelumnya, model KNN memperoleh nilai Accuracy tertinggi sebesar 95,93%, diikuti oleh Decision Tree sebesar 94,78%, sedangkan Gaussian Naive Bayes memperoleh nilai terendah sebesar 62,28%. Hasil visualisasi ini menunjukkan bahwa model KNN memiliki tingkat ketepatan prediksi keseluruhan paling tinggi dibandingkan kedua model lainnya.

3.5 Pelatihan dan Evaluasi Model Model Linear Regression

Selain melakukan klasifikasi status diabetes, penelitian ini juga menerapkan Linear Regression untuk memprediksi nilai `blood_glucose_level`. Pada proses regresi, atribut `blood_glucose_level` digunakan sebagai variabel target, sedangkan atribut `diabetes` tidak disertakan sebagai fitur. Dengan demikian, model memprediksi kadar glukosa darah berdasarkan atribut lain, yaitu jenis kelamin, usia, hipertensi, penyakit jantung, riwayat merokok, BMI, dan HbA1c.

In [18]:
```
# Siapkan fitur regresi dan target numerik
X_reg = data.drop(columns=["blood_glucose_level", "diabetes"])
y_reg = data["blood_glucose_level"]
# Identifikasi kolom kategorikal dan numerik
cat_reg = X_reg.select_dtypes(include="object").columns.tolist()
num_reg = X_reg.select_dtypes(exclude="object").columns.tolist()
# Preprocessing data regresi
preprocessor_reg = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median"))
    ]), num_reg),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(
            handle_unknown="ignore",
            sparse=False
        ))
    ]), cat_reg)
])
# Membagi data latih dan data uji
Xr_train, Xr_test, yr_train, yr_test = train_test_split(
    X_reg,
    y_reg,
    test_size=0.20,
    random_state=42
)
# Membuat model regresi linear
linear_model = Pipeline([
    ("preprocessing", preprocessor_reg),
    ("model", LinearRegression())
])
# Melatih model
linear_model.fit(Xr_train, yr_train)
# Melakukan prediksi
yr_pred = linear_model.predict(Xr_test)
# Evaluasi model
regression_results = pd.DataFrame({
    "Metrik": ["R2", "MAE", "RMSE"],
    "Nilai": [
        r2_score(yr_test, yr_pred),
        mean_absolute_error(yr_test, yr_pred),
        np.sqrt(mean_squared_error(yr_test, yr_pred))
    ]
})
display(regression_results.round(4))
#202431124_Abhillasitta
```

Pada tahap awal, X_reg ditetapkan sebagai variabel fitur dengan menghapus kolom blood_glucose_level dan diabetes, sedangkan y_reg ditetapkan sebagai variabel target yang berisi nilai kadar glukosa darah. Penghapusan kolom diabetes bertujuan agar model tidak menggunakan status diabetes sebagai salah satu variabel prediktor.

Selanjutnya, dilakukan identifikasi atribut kategorikal dan numerik menggunakan select_dtypes(). Proses preprocessing diterapkan melalui ColumnTransformer, yaitu menggunakan SimpleImputer(strategy="median") untuk menangani kemungkinan nilai kosong pada atribut numerik dan SimpleImputer(strategy="most_frequent") untuk atribut kategorikal. Atribut kategorikal kemudian diubah menjadi bentuk numerik menggunakan OneHotEncoder agar dapat diproses oleh algoritma regresi. Setelah itu, dataset dibagi menjadi 80% data latih dan 20% data uji menggunakan train_test_split(), dengan random_state=42 agar pembagian data dapat direproduksi secara konsisten.

Tahap berikutnya adalah membangun model Linear Regression menggunakan Pipeline yang menggabungkan proses preprocessing dan algoritma LinearRegression(). Model kemudian dilatih menggunakan linear_model.fit(Xr_train, yr_train) untuk mempelajari hubungan antara atribut prediktor dan kadar glukosa darah. Setelah proses pelatihan selesai, linear_model.predict(Xr_test) digunakan untuk menghasilkan prediksi berdasarkan

data uji. Hasil prediksi tersebut selanjutnya dievaluasi menggunakan tiga metrik, yaitu R^2 (R-Squared) untuk mengukur kemampuan model dalam menjelaskan variasi nilai target, MAE (Mean Absolute Error) untuk menghitung rata-rata selisih absolut antara nilai aktual dan prediksi, serta RMSE (Root Mean Squared Error) untuk mengukur rata-rata kesalahan prediksi dengan memberikan penalti lebih besar terhadap kesalahan yang tinggi. Seluruh hasil evaluasi disusun dalam DataFrame regression_results dan ditampilkan menggunakan display(regression_results.round(4)) agar nilai evaluasi tersaji dengan empat angka di belakang koma.

	Metrik	Nilai
0	R2	0.0478
1	MAE	30.9012
2	RMSE	39.8665

Abhilla_202431124

1. R^2 (R-Squared) = 0.0478 (4,78%): Nilai koefisien determinasi sebesar 0,0478 menunjukkan bahwa variabel prediktor (seperti usia, BMI, HbA1c, hipertensi, penyakit jantung, jenis kelamin, dan riwayat merokok) hanya mampu menjelaskan 4,78% variasi dari kadar glukosa darah. Sisanya sebesar 95,22% dipengaruhi oleh faktor lain di luar model. Hal ini mengindikasikan bahwa hubungan antara fitur prediktor tersebut dengan kadar glukosa darah bersifat sangat non-linear dan kurang tepat jika hanya dimodelkan menggunakan regresi linear standar.
2. MAE (Mean Absolute Error) = 30.9012: Nilai rata-rata selisih mutlak antara kadar glukosa darah aktual dengan nilai hasil prediksi model adalah sebesar 30,9012 mg/dL. Artinya, secara rata-rata, prediksi model menyimpang sekitar 30,9 unit dari angka glukosa darah yang sebenarnya.
3. RMSE (Root Mean Squared Error) = 39.8665: Nilai akar rata-rata kuadrat kesalahan sebesar 39,8665 mg/dL mengukur tingkat penyimpangan dengan memberikan penalti lebih besar pada kesalahan prediksi yang bernilai tinggi. Perbedaan antara nilai RMSE dan MAE menunjukkan adanya beberapa data outlier yang memiliki deviasi prediksi cukup besar.

BAB IV
PENUTUP

Berdasarkan hasil analisis, pengolahan data, serta pengujian model Machine Learning yang telah dilakukan pada dataset 02_diabetes.csv, diperoleh beberapa kesimpulan sebagai berikut:

1. Karakteristik & Kebersihan Dataset:
    - Dataset awal memiliki 100.000 baris data dan 9 kolom atribut. Setelah dilakukan tahap preprocessing, sebanyak 3.854 baris data duplikat berhasil dihapus, menghasilkan 96.146 baris data bersih yang siap diolah.
    - Dataset mengalami kondisi class imbalance yang cukup signifikan pada variabel target diabetes, dengan proporsi 91,2% individu tidak menderita diabetes dan 8,8% menderita diabetes.
    - Berdasarkan analisis korelasi Pearson, fitur blood_glucose_level ($r = 0.420$) dan HbA1c_level ($r = 0.401$) memiliki korelasi linear positif paling kuat terhadap status diabetes dibandingkan atribut numerik lainnya.
2. Performa Model Klasifikasi:
    - K-Nearest Neighbors (KNN, $k = 5$): Menjadi algoritma terbaik dalam klasifikasi keseluruhan dengan Accuracy tertinggi (95,93%), Precision (88,52%), F1-Score (72,82%), dan ROC-AUC (90,08%). Model ini sangat tepat digunakan jika fokus utama adalah meminimalkan kesalahan prediksi positif (False Positive).
    - Decision Tree: Menghasilkan performa yang cukup seimbang dengan Accuracy (94,78%), Recall (74,17%), dan F1-Score (71,50%). Algoritma ini memberikan keunggulan interpretabilitas aturan keputusan yang jelas.
    - Gaussian Naive Bayes: Memperoleh Recall tertinggi (96,17%) dan ROC-AUC (90,65%), namun memiliki Accuracy (62,28%) dan Precision (18,49%) yang rendah. Model ini sangat sensitif dan mampu menjaring hampir seluruh penderita diabetes, tetapi menghasilkan banyak prediksi false positive.
3. Penerapan Model Regresi Linear:
    - Algoritma Linear Regression berhasil diterapkan untuk memprediksi variabel kontinu blood_glucose_level berdasarkan 7 atribut prediktor (tanpa

melibatkan variabel diabetes). Evaluasi dilakukan menggunakan metrik R^2, MAE, dan RMSE untuk mengukur tingkat presisi dan kesalahan estimasi nilai glukosa darah.

DAFTAR PUSTAKA

Açıkkar, M., & Tokgöz, S. (2023). An improved KNN classifier based on a novel weighted voting function and adaptive k-value selection. Neural Computing and Applications. https://doi.org/10.1007/s00521-023-09272-8

Çetin, A. İ., & Büyüklü, A. H. (2025). A new approach to K-nearest neighbors distance metrics on sovereign country credit rating. Kuwait Journal of Science, 52, 100324. https://doi.org/10.1016/j.kjs.2024.100324

Hasan, S. M. M., Uddin, M. P., Mamun, M. A., Sharif, M. I., Ulhaq, A., & Krishnamoorthy, G. (2023). A machine learning framework for early-stage detection of autism spectrum disorders. IEEE Access. https://doi.org/10.1109/ACCESS.2022.3232490

Ilyas, I., Zafar, A., Afzal, M. T., et al. (2022). Advanced machine learning modeling approach for prediction of compressive strength of FRP confined concrete using multiphysics genetic expression programming. Polymers, 14, 1789. https://doi.org/10.3390/polym14091789

Madyatmadja, E. D., Rianto, A., & Andry, J. F. (2021). Analysis of big data in healthcare using decision tree algorithm. In 2021 1st International Conference on Computer Science and Artificial Intelligence (ICCSAI). 10.1109/ICCSAI53272.2021.9609734

Marudi, M., Ben-Gal, I., & Singer, G. (2022). A decision tree-based method for ordinal classification problems. IISE Transactions. https://doi.org/10.1080/24725854.2022.2081745

Ozaltin, O., Coskun, O., Yeniay, O., & Subasi, A. (2022). A deep learning approach for detecting stroke from brain CT images using OzNet. Bioengineering, 9, 783. https://doi.org/10.3390/diagnostics12010116

Singh, V., Asari, V. K., & Rajasekaran, R. (2022). A deep neural network for early detection and prediction of chronic kidney disease. Diagnostics, 12, 116. https://doi.org/10.3390/bioengineering9120783